Machine Learning Book Recommendation System

Project

Shashwat Gangwar
Sumit Rawat
Samyak Jain
Sachin Srivastava
Kshitij Varma

Department of Computer Science

Table of Contents

1. Intro duction
2. System Architecture & Tech Stack
3. Data Flow & State Management
4. Mathematical Methodology
5. Future Enhancements

Introduction: The Problem

- Information Overload: With millions of titles available, manual book discovery is overwhelming for users.
- Static Libraries: Traditional cataloging systems lack personalization and fail to adapt to changing user preferences.
- The ‘Cold Start’ Issue: A fundamental challenge in recommendation engines where new users with no history cannot receive personalized suggestions.

Introduction: The Solution

- Hybrid Recommendation Engine:
  - Popularity-Based Filtering: Highlights globally top-rated books to effectively bypass the cold start problem for new users.
  - Item-Item Collaborative Filtering: Analyzes rating patterns to provide highly personalized recommendations for established users.
- Web Application Web Interface: A responsive, user-friendly platform for seamless discovery, rating, and tracking.

System Architecture & Tech Stack

- Frontend Presentation:
  - HTML5 & Bootstrap 5: Provides a responsive, mobile-first UI.
  - Jinja2: Enables dynamic template rendering on the server side.
- Backend Application:
  - Python & Flask: Serves as the lightweight, core routing framework.
  - Gunicorn: Acts as the robust WSGI HTTP server for production.
- Data Layer:
  - SQLite (SQLAlchemy ORM): Manages transactional, fresh data.
  - Pandas: Handles historical CSV data manipulation and matrix op erations.
- Deployment: Hosted on an GCP Virtual Machine.

Data Flow & API Integration

- Real-Time Interactions: User ratings, reviews, and search queries are processed instantly to maintain a responsive UI.
- Data Ingestion:
  - System interfaces with the Google Books API to fetch metadata for new, uncataloged books.
  - Parsed JSON responses are serialized and saved to the persistent SQLite database.
- State Persistence: SQLAlchemy ensures ACID compliance so user profiles and newly fetched books are reliably stored.

Asynchronous Pipeline

- Challenge: Recomputing the recommendation matrix on every user rating blocks the Flask server, causing UI latency.
- Solution (retrain.py): An asynchronous batch-processing pipeline that runs independently of the web thread.

Batch Processing Concept (retrain.py)

```
def batch_train():
    # Load historical CSVs and new SQLite entries
    df_history = pd.read_csv('historical_ratings.csv')
    df_new= fetch_new_ratings_from_sqlite()

    # Merge and recompute similarity matrix
    merged_df= pd.concat([df_history, df_new])
    pt = merged_df.pivot_table(index='Book-Title',
                               columns='User-ID',
                               values='Book-Rating')
    compute_cosine_sim(pt)
```

Methodology: Collaborative Filtering

- Core Concept: Recommends books mathematically similar to those a user has already rated highly.
- Vector Space Model: Every book is represented as an n-dimensional vector, where n is the number of users, and the vector components are the ratings given by those users.
- Metric: We utilize Cosine Similarity to measure the cosine of the angle between two vectors. This isolates the orientation (rating pattern similarity) regardless of magnitude (overall number of ratings).

Similarity Formula

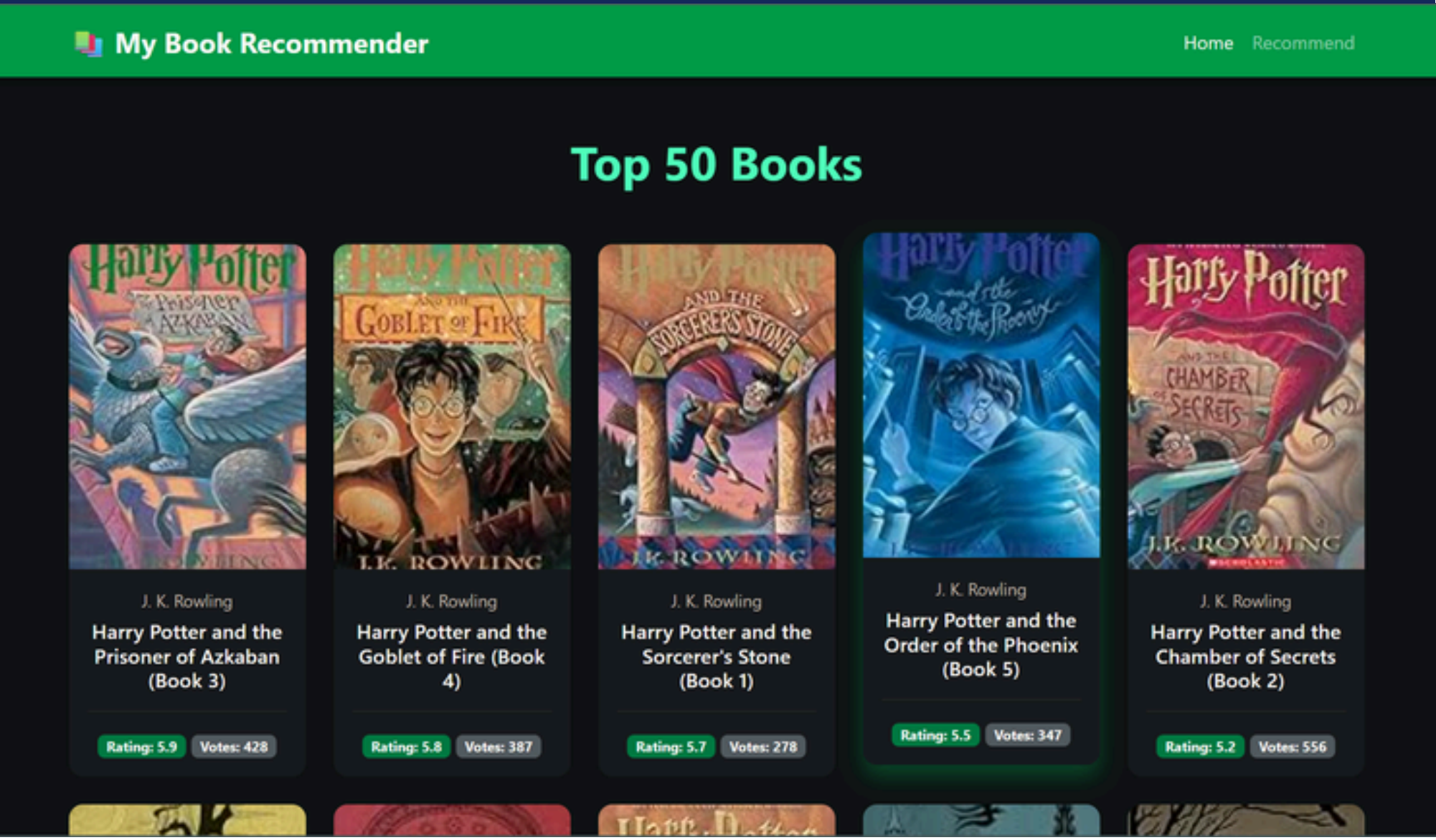
book vectors **A** and **B**:

$$\text{sim}(\mathbf{A}, \mathbf{B}) = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$

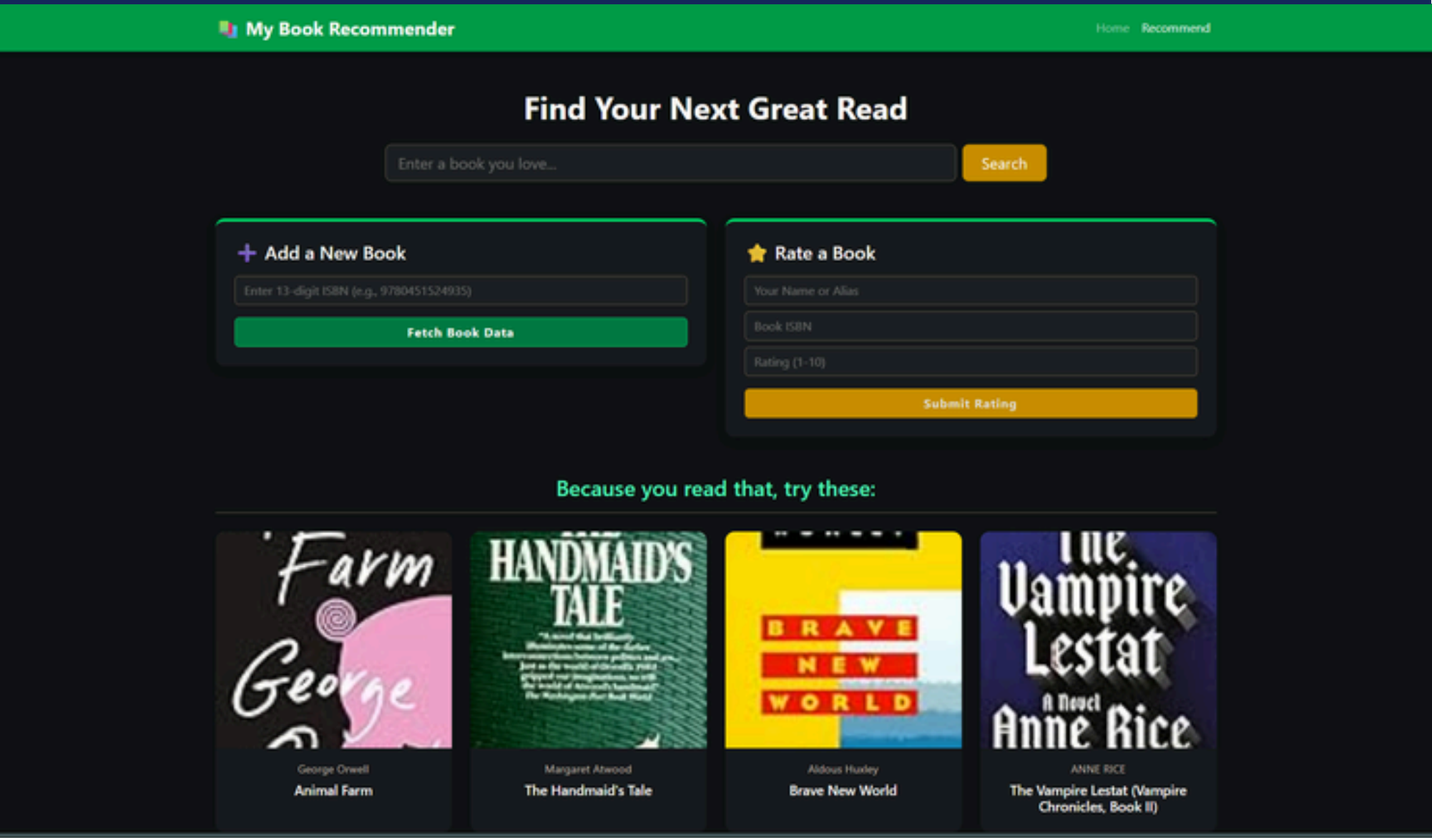
Future Enhancements

1. Cloud Database Migration:
   - Transitioning from SQLite to PostgreSQL (e.g., AWS RDS) to handle high-concurrency read/write operations and distributed scaling.
2. Robust Security:
   - Implementing User Authentication (OAuth 2.0 or JWT) to securely encapsulate personal data and user rating histories.
3. Content-Based NLP Integration:
   - Adding NLP-based Content Filtering (e.g., applying TF-IDF or BERT embeddings to book summaries).
   - This will solve the item-side cold start problem, allowing the system to recommend newly added books before they receive user ratings.

Working Screenshot



Working Screenshot



Thank You